RICHFIELD
richfield.ac.za

Name and Surname: Caleb Singh

Student ITS No: 402306657

Qualification: Bachelor of Science in IT **Year of Study:** 3 **Semester:** 2

Assignment due date: _____ **Date submitted:** 23 October 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: _____ **Date:** _____

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Pantry and Plate Smart Pantry Manager

Mobile App Development 700 practical assignment report

Caleb Singh
Student number 402306657
23 October 2026

Pantry & Plate is a Java Android application for tracking leftover ingredients and finding meals that can be prepared with the quantities already available. This report explains its design, SQLite data model, implementation and verification.

Repository: https://github.com/calebsingh23-afk/mad_assignment

Assistance acknowledgement: OpenAI Codex assisted with implementation, testing and report preparation. The declaration on the supplied cover page is reserved for the student to review and sign after checking the work and the institution's academic integrity requirements.

Table of contents

Introduction . . . 3
System design Screen flow . . . 4
System design Database model . . . 5
System design Visual choices . . . 6
Screenshots of the running application 1 . . . 7
Screenshots of the running application 2 . . . 8
Screenshots of the running application 3 . . . 9
Screenshots of the running application 4 . . . 10
Screenshots of the running application 5 . . . 11
Screenshots of the running application 6 . . . 12
Screenshots of the running application 7 . . . 13
Screenshots of the running application 8 . . . 14
Key code snippets 1 . . . 15
Key code snippets 2 . . . 16
Challenges and solutions . . . 17
Verification and requirement coverage . . . 18
Conclusion and reflection . . . 19
References . . . 20

Introduction

Leftovers become waste when people forget what they have, overlook expiry dates or choose meals that need ingredients they do not own. Pantry & Plate is intended for students and households that want to make practical meals from food already in their kitchen.

The app keeps an editable pantry and compares it with 20 locally stored recipes. A recipe is suggested only when all requirements are met in sufficient quantities. Even water and oil must be recorded when a recipe requires them. Partial matches are excluded.

The scope includes pantry creation, reading, updating and deletion, quantity and date validation, strict suggestions, recipe details and an expiry badge setting. SQLite supports an offline experience. The app has no location, mapping, payment or network permissions.

The implementation uses Java Activities, explicit Intents, LinearLayout containers and custom ListView adapters. The design follows the Android component model, with data refreshed when list Activities resume (Google, 2026a; Google, 2026b).

System design Screen flow

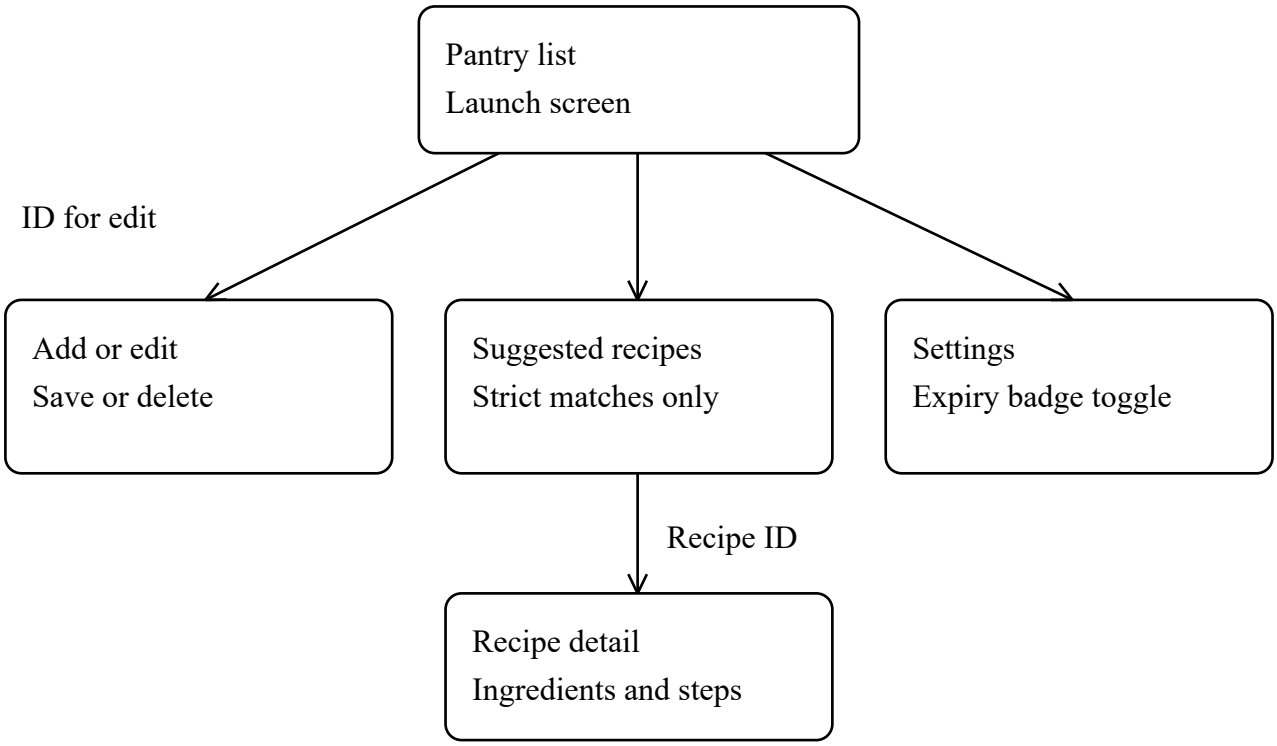


Figure 1. Five distinct Activities. Pantry, Recipes and Settings are available through the bottom navigation. Add and edit use the same form Activity. Ingredient and recipe database IDs are passed as Intent extras.

Returning after a save or deletion causes PantryActivity.onResume to reload persisted rows. RecipesActivity also recalculates suggestions on resume. Explicit Intents identify the destination Activity and pass IDs rather than full mutable records (Google, 2026b).

The presentation layer delegates persistence to Database and decision rules to Matcher. BaseActivity runs database operations on a single background executor and delivers results to the main thread. This keeps SQLite work out of UI rendering (Google, 2026c).

System design Database model

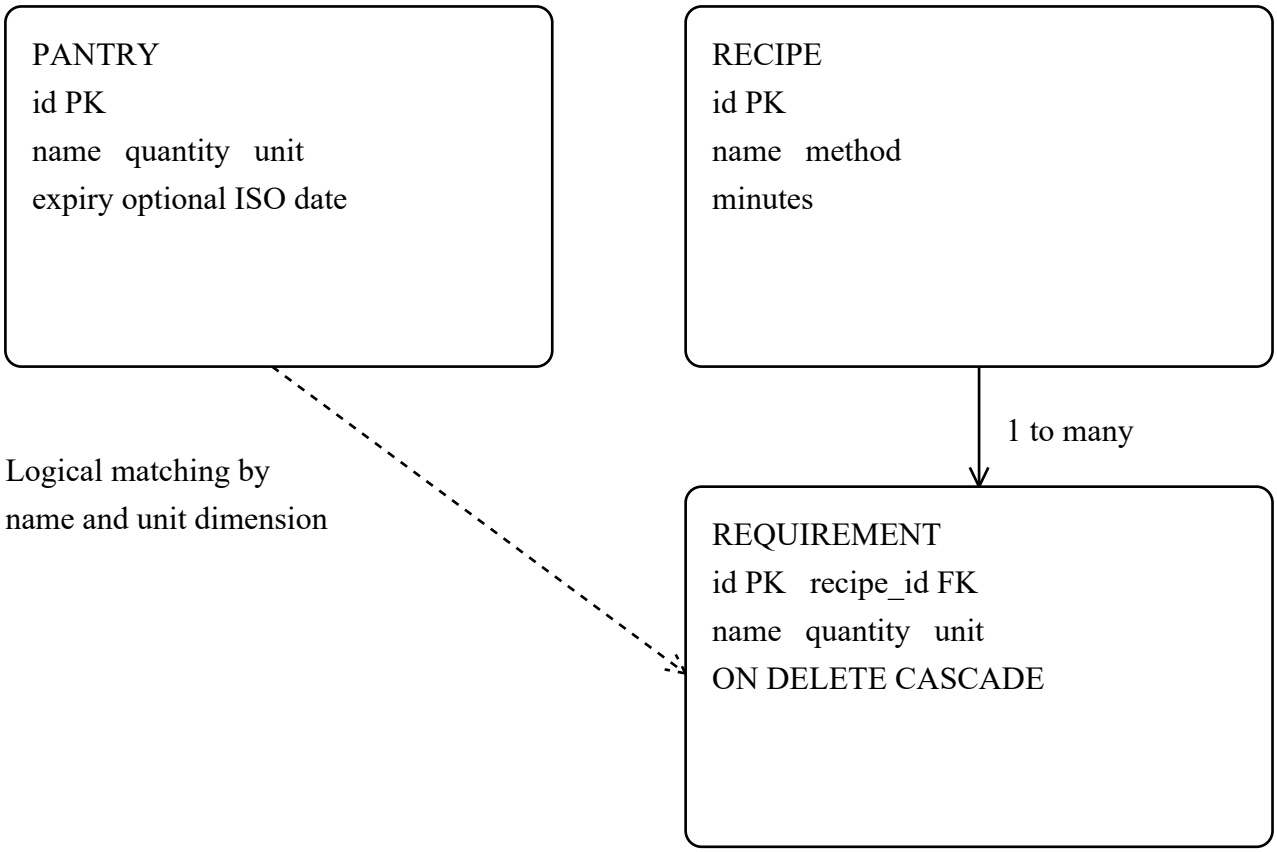


Figure 2. SQLite tables and relationships. Recipe has many requirement rows through recipe_id. Pantry has no foreign key to a recipe because the matching algorithm compares normalized names and compatible unit dimensions.

Pantry stores id, name, positive quantity, unit and optional expiry date. Recipe stores id, name, preparation method and minutes. Requirement stores id, recipe_id, ingredient name, positive quantity and unit. Foreign keys and quantity CHECK constraints protect the stored model.

SQLiteOpenHelper creates and seeds the database on first use. Later opens reuse it without duplicating recipes. ContentValues and selection arguments support safe writes. Cursor resources are closed with try-with-resources. No destructive upgrade is implemented for this version; a future schema change needs an explicit migration (Google, 2026c).

System design Visual choices

The app combines a warm cream background (#F7F3E8), forest green actions (#235345), dark text (#1D3129) and restrained terracotta accents (#AB4D30). This palette relates to food and plant life while keeping the interface calm and legible. Information is also stated in text, so expiry meaning does not depend on colour alone.

Serif screen titles establish a distinctive editorial character, while sans serif labels and form controls support quick reading. Rounded white cards group pantry entries and recipes. Spacious controls and a persistent bottom navigation make the main tasks easy to find.

The pantry sorts dated items first, placing undated items afterwards. Expired items remain visible for management but never qualify for recipe matching. Optional badges identify items expiring within three days. These badges are local interface feedback rather than scheduled notifications.

The form names each input and displays errors beside the relevant field. Delete requires confirmation. Empty pantry and zero-match recipe screens explain the next step, rather than leaving an unexplained blank screen.

Responsive vertical layouts and scrolling forms support smaller displays. Android 15 system-bar insets are applied to prevent content from being obscured. The evidence images that follow are captures of the actual installed Android application.

Screenshots of the running application 1

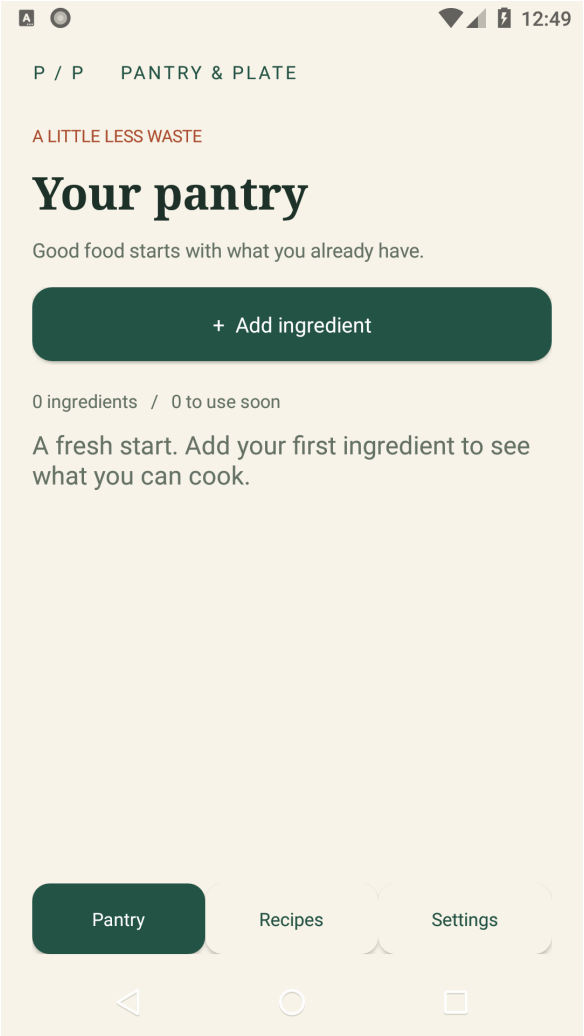


Figure 3. Fresh installation with an empty SQLite pantry and a clear invitation to add an ingredient.

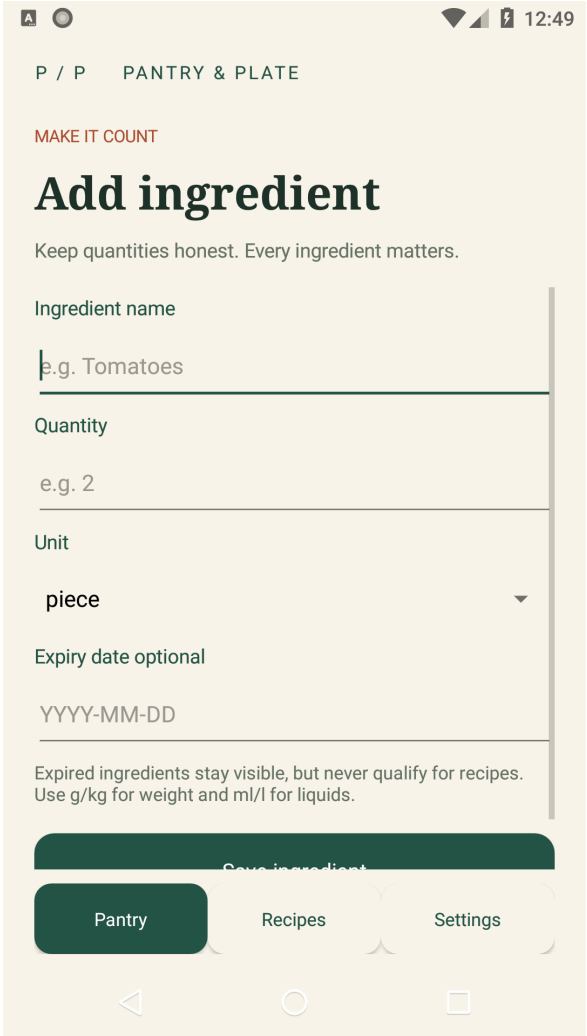


Figure 4. Add ingredient form with name, quantity, unit and optional expiry date.

Screenshots of the running application 2

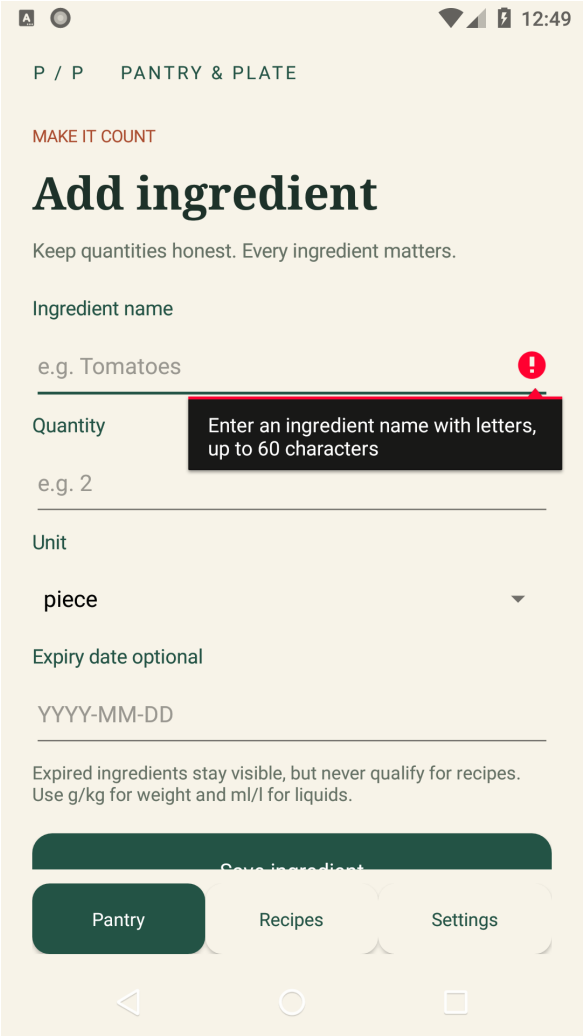


Figure 5. A rejected input shows a field-level validation message.

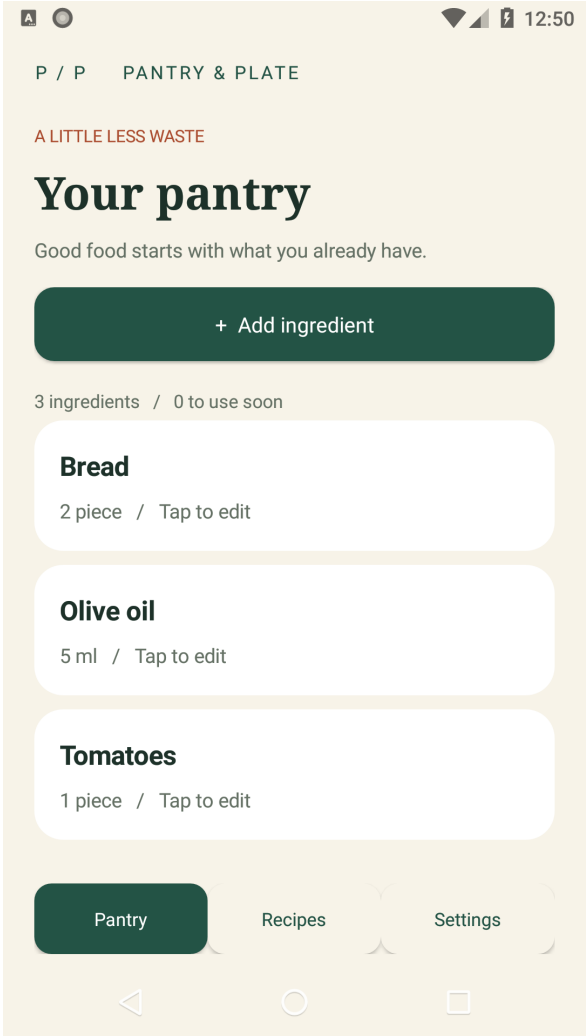


Figure 6. Saved ingredients are read from SQLite and displayed through the custom pantry adapter.

Screenshots of the running application 3

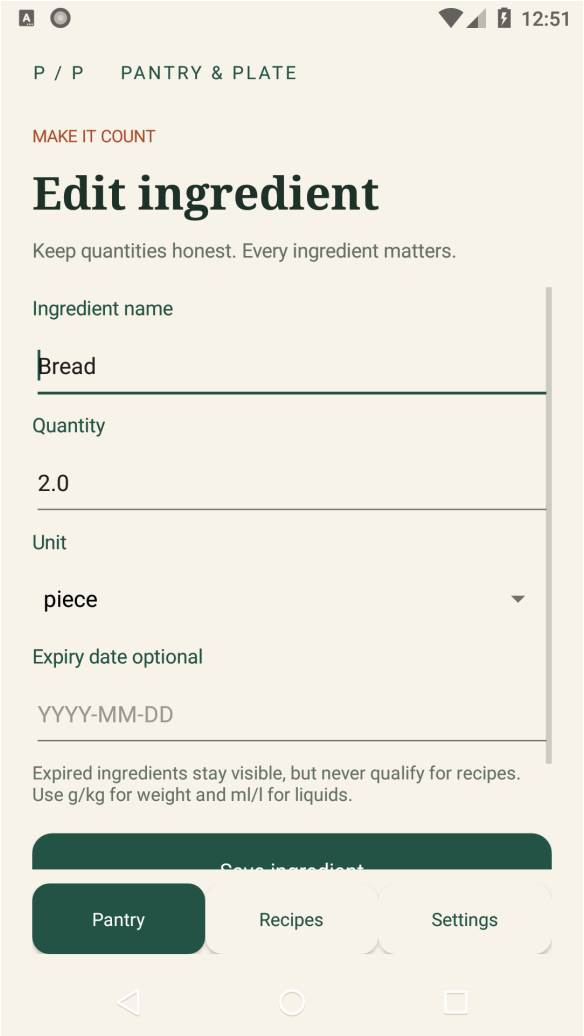


Figure 7. An existing ingredient is loaded by its Intent ID for editing.

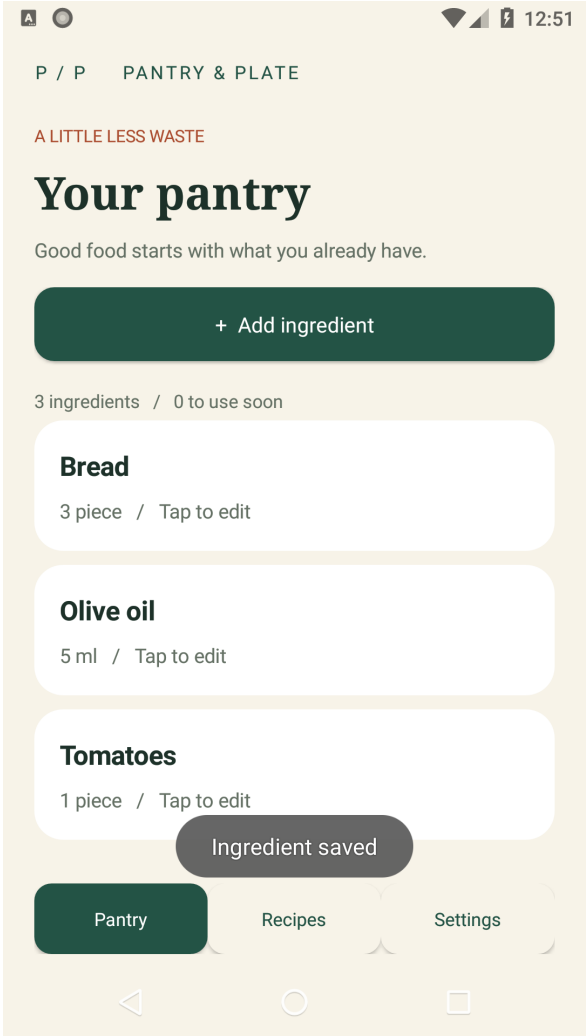


Figure 8. The list reflects a saved quantity change.

Screenshots of the running application 4

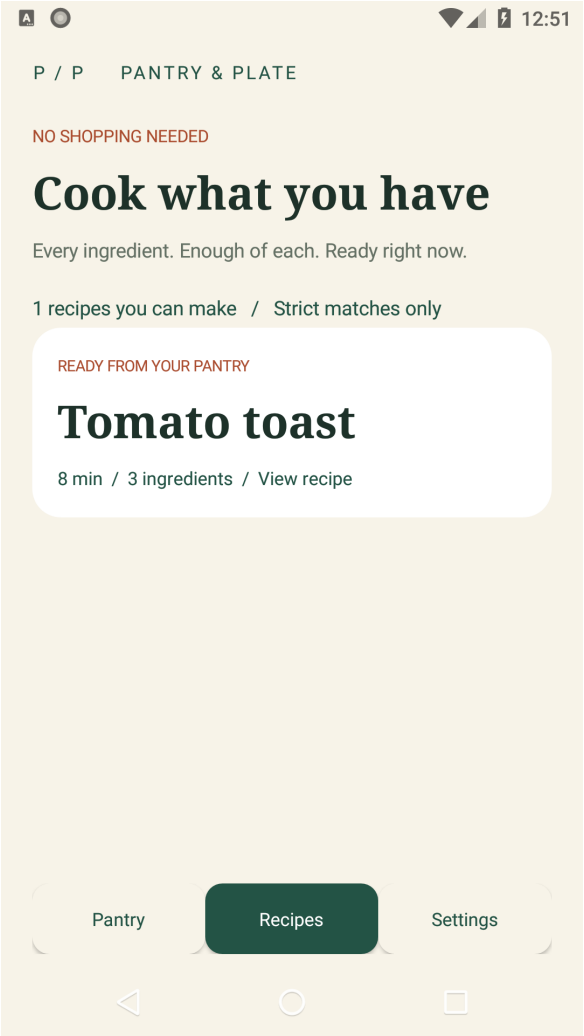


Figure 9. Strict suggestions show recipes for which every requirement is available.

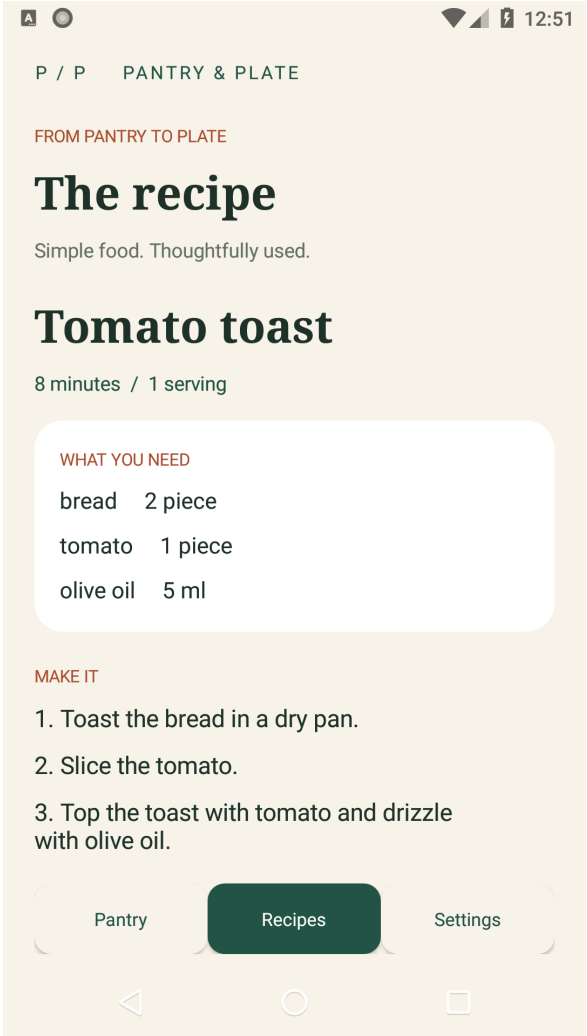


Figure 10. Recipe detail shows every required ingredient and numbered preparation steps.

Screenshots of the running application 5

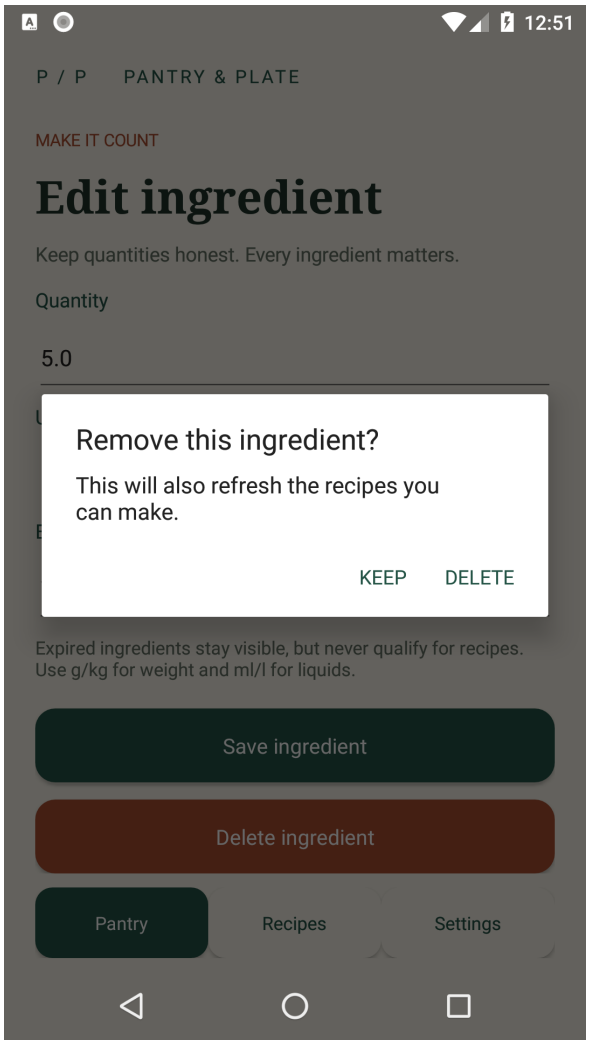


Figure 11. A confirmation dialog protects against accidental deletion.

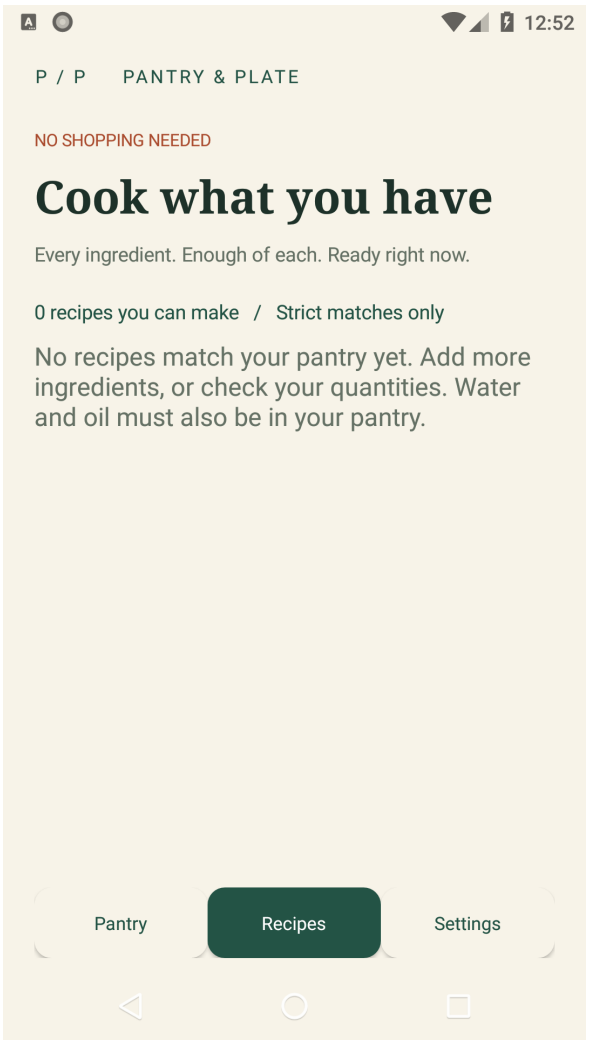


Figure 12. Removing a required ingredient excludes the recipe and displays helpful zero-match feedback.

Screenshots of the running application 6

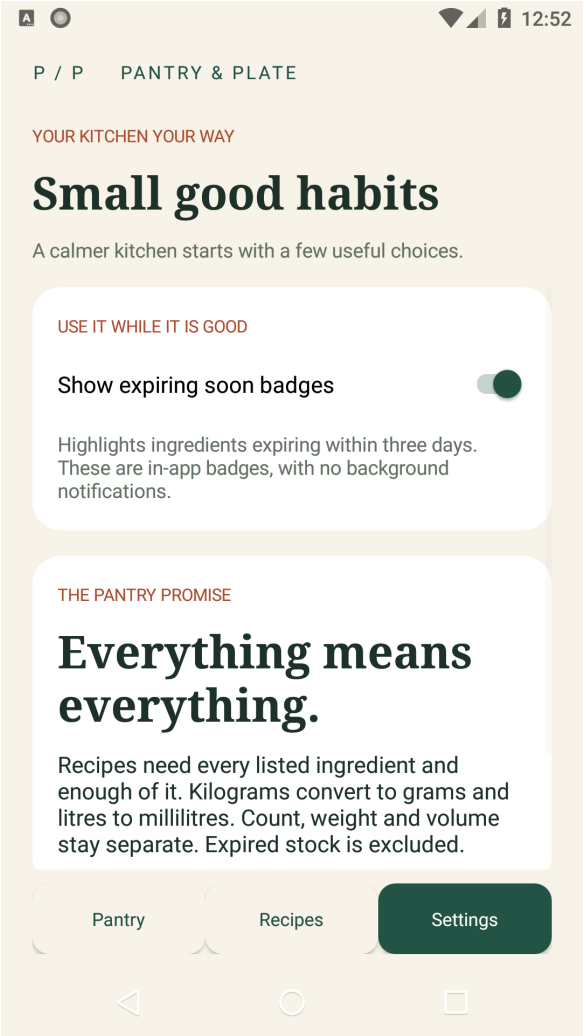


Figure 13. The settings screen explains matching and provides a persistent expiry badge preference.

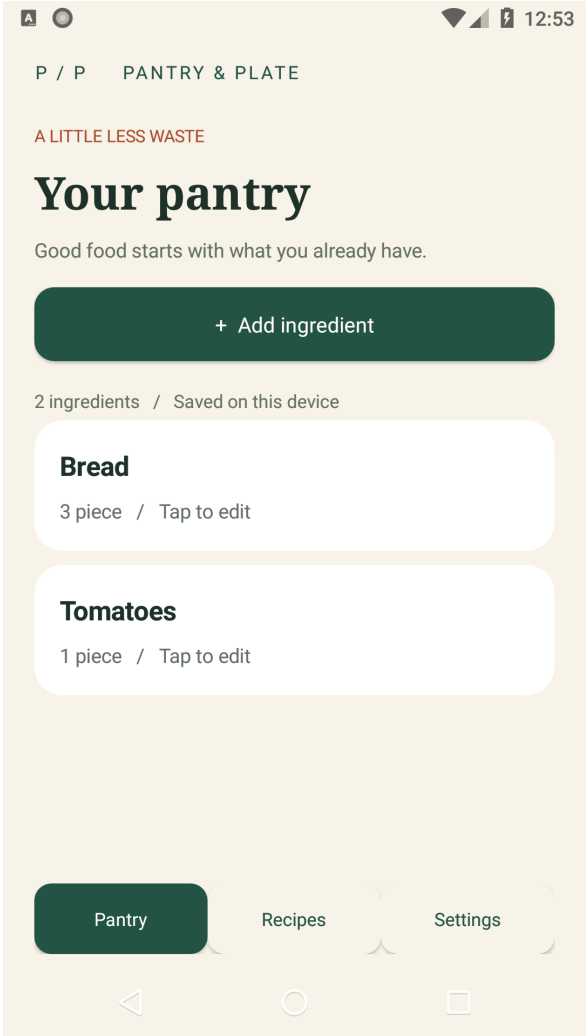


Figure 14. After a force-stop and relaunch, the pantry records remain stored on the device.

Screenshots of the running application 7

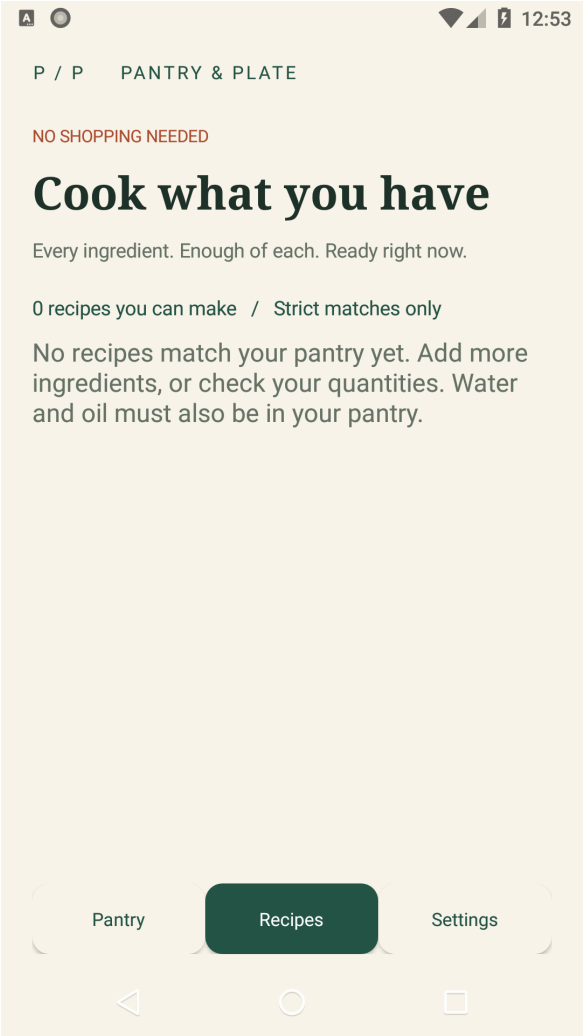


Figure 15. All three ingredient names are present, but only one bread piece is available. Tomato toast requires two, so it is excluded.

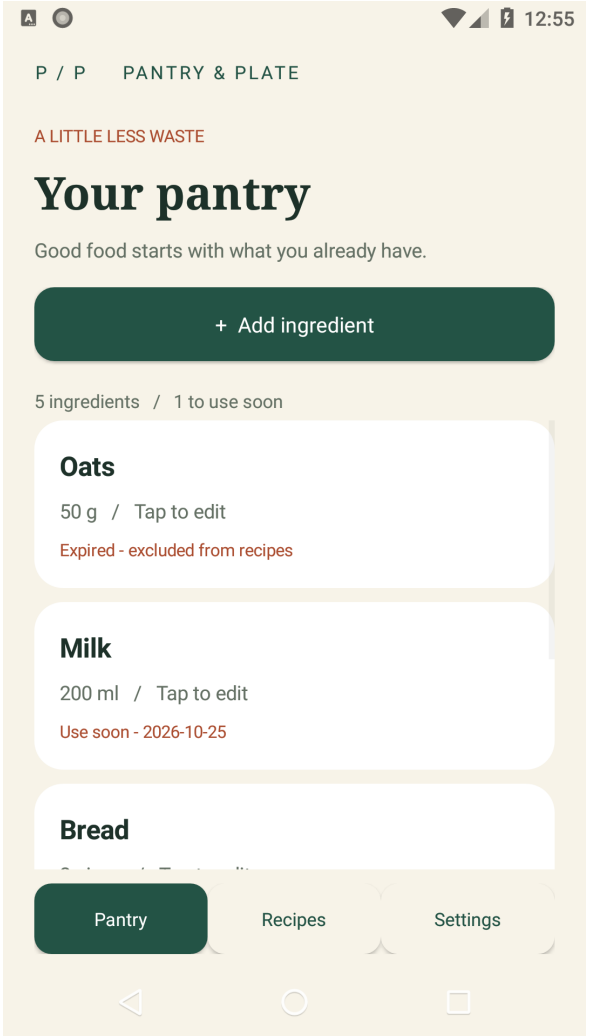


Figure 16. Expiry badges distinguish expired stock from food to use soon. Expired ingredients are excluded from suggestions.

Screenshots of the running application 8

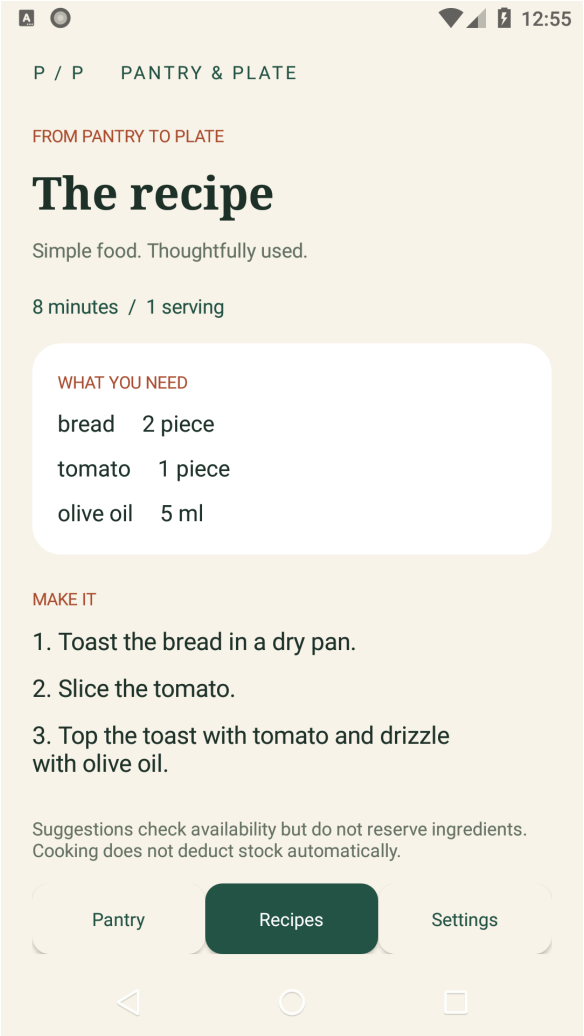


Figure 17. Scrolling the recipe detail reveals the complete preparation method and the note about stock deduction.

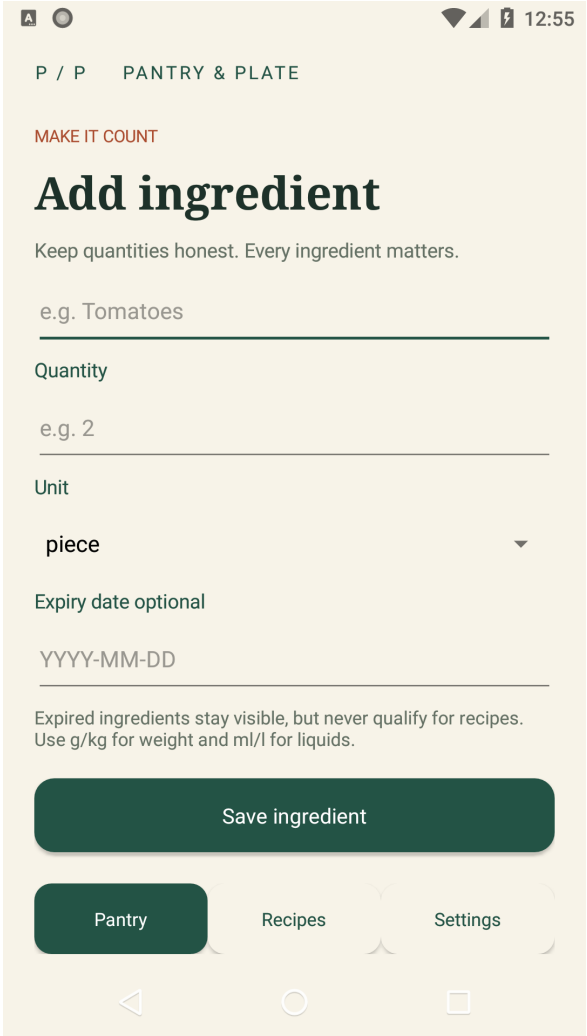


Figure 18. Scrolling the ingredient form reveals the complete save action and the guidance about units and expiry.

Key code snippets 1

Matcher.java lines 68 to 72

```
  if (required.isEmpty()) return false;
  for (Map.Entry<String, Double> entry : required.entrySet())
   if (available.getOrDefault(entry.getKey(), 0.0) + 1e-9 < entry.getValue()) return false;
  return true;
 }
```

Strict matching aggregates stock and recipe requirements by canonical name and unit dimension. Expired rows are skipped. The final comparison rejects the whole recipe if any requirement exceeds available stock.

Database.java lines 40 to 50

```
 public long save(Ingredient item) {
  ContentValues values = new ContentValues();
  values.put("name", item.name);
  values.put("quantity", item.quantity);
  values.put("unit", item.unit);
  values.put("expiry", item.expiry);
  if (item.id == 0) return getWritableDatabase().insertOrThrow("pantry", null, values);
  if (getWritableDatabase().update("pantry", values, "id=?", new String[] {"" + item.id}) !=
1)
    throw new IllegalStateException("Ingredient no longer exists");
  return item.id;
 }
```

Pantry persistence uses ContentValues. New rows receive a SQLite ID; existing rows are updated using a parameterized ID selection. Updating a missing row raises an error.

Key code snippets 2

PantryAdapter.java lines 43 to 47

```
  Ingredient i = getItem(position);
  TextView name = Ui.text(context, i.name, 19, Ui.INK);
  name.setTypeface(null, Typeface.BOLD);
  card.addView(name);
  card.addView(Ui.text(context, Ui.amount(i) + "   /   Tap to edit", 14, Ui.MUTED));
```

The custom adapter turns database models into pantry cards. It reuses a LinearLayout where possible and rebuilds its child views to avoid stale content. It displays the stored quantity and expiry feedback.

PantryActivity.java lines 10 to 13

```
protected void onResume() {
  super.onResume();
  render();
}
```

The lifecycle callback refreshes the pantry after returning from an editor. Reading persistent data again avoids an outdated list after a change.

Challenges and solutions

Quantity matching across different units was a core correctness challenge. Comparing raw numbers could treat 0.5 kg as less than 100 g or incorrectly allow grams to stand in for millilitres. Matcher converts mass and volume separately and preserves the count dimension. Regression tests cover both valid conversions and invalid cross-dimension substitutions.

Duplicate pantry entries can cause false shortages if each row is tested alone. Conversely, repeated requirements can undercount demand if each is checked independently. The solution aggregates both sides before comparing totals. Tests verify combined stock and repeated requirements.

Toolchain setup encountered stalled Android SDK downloads and connection resets. Official package downloads were retried with bounded transfers. The separate pure Java matcher harness allowed the main business rule to be checked while the Android build environment was being prepared. The GitHub push also encountered an account permission mismatch. Signing into the repository owner account resolved it, and the incremental history was pushed successfully.

Verification and requirement coverage

Build: Gradle 8.9, Android Gradle Plugin 8.7.3, SDK 35 and build tools 35.0.0. assembleDebug and assembleDebugAndroidTest passed.
Device: Android 8.0 API 26 x86_64 emulator, 1080 by 1920, WHPX acceleration.
Matcher: all 17 pure Java checks passed.
SQLite instrumentation: create, read, update, delete, close and reopen persistence, 20 complete recipes and idempotent seed passed.
UI evidence: actual emulator captures document the five screens, validation, CRUD, strict matching and process relaunch.
Scope of verification: runtime checks were performed on API 26. Android 15 inset handling is implemented but was not runtime-tested on API 35.
GitHub: public repository confirmed; incremental history pushed successfully and remote main verified against local HEAD.

The project includes an executable pure Java regression harness under tests and an Android instrumentation runner under app/src/androidTest. The former checks matching independently of Android, while the latter checks SQLite CRUD and persistence against the device database.

The screen evidence covers entry, list display, validation, update, deletion, strict suggestions, detail, settings and relaunch persistence. Matching does not reserve stock between recipes or automatically deduct food after cooking. These are defined limitations rather than hidden inventory changes.

Version control records incremental implementation stages with descriptive messages and real timestamps. All work in this assisted session happened in one sitting. This history is not evidence of development spread over several days.

Conclusion and reflection

Pantry & Plate provides a focused offline implementation of the Smart Pantry Manager brief. It connects pantry CRUD, persisted SQLite data, strict quantity-aware matching, recipe details and settings through five Java Activities.

The implementation illustrates why business rules should be isolated from interface code: Matcher can be tested using ordinary Java while Android-specific persistence and navigation are checked on a device. Unit dimensions, duplicate stock and expiry rules require explicit decisions to prevent misleading suggestions.

Further work could introduce ingredient selection from a controlled vocabulary, more aliases, nutrition and allergy metadata, configurable recipe servings and non-destructive schema migrations. Any added feature should preserve the rule that suggested recipes require every ingredient in a sufficient quantity.

Before recording the demonstration, the student should rehearse the code explanations, repeat the CRUD and missing-ingredient scenarios, and adapt the reflection to their own understanding. The video is intentionally supplied by the student and is not included in this prepared package.

References

Google (2026a) The activity lifecycle. Android Developers. Available at: https://developer.android.com/guide/components/activities/activity-lifecycle (Accessed: 23 October 2026).

Google (2026b) Intents and intent filters. Android Developers. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 23 October 2026).

Google (2026c) SQLiteOpenHelper. Android Developers. Available at: https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper (Accessed: 23 October 2026).

Google (2026d) BaseAdapter. Android Developers. Available at: https://developer.android.com/reference/android/widget/BaseAdapter (Accessed: 23 October 2026). Custom list display follows this adapter contract.

Richfield Graduate Institute of Technology (n.d.) Mobile App Development 700 Practical Assignment Smart Pantry Manager. Supplied assessment brief.

OpenAI (2026) Codex. AI assistance for source implementation, verification and documentation during this project session. Outputs reviewed against the supplied brief.